

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_ARGS 50

// Token Types
typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,          // |
    TOKEN_REDIRECT_IN,   // <
    TOKEN_REDIRECT_OUT   // >
} TokenType;

// Token Structure
typedef struct {
    char *value;
    TokenType type;
} Token;

// Execution Structure (Parsed Command)
typedef struct {
    char *args[MAX_ARGS];
    int arg_count;
    char *input_file;
    char *output_file;
    int has_pipe;
} Command;
```

```
// 1. TOKENIZER: Splits input into tokens based on whitespace and special delimiters
int tokenize(const char *input, Token tokens[]) {
    int token_count = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Handle whitespace
        if (isspace(input[i])) {
            i++;
            continue;
        }

        // Identify Delimiters
        if (input[i] == '|') {
            tokens[token_count].value = strdup("|");
            tokens[token_count].type = TOKEN_PIPE;
            token_count++;
            i++;
        } else if (input[i] == '<') {
            tokens[token_count].value = strdup("<");
            tokens[token_count].type = TOKEN_REDIRECT_IN;
            token_count++;
            i++;
        } else if (input[i] == '>') {
            tokens[token_count].value = strdup(">");
            tokens[token_count].type = TOKEN_REDIRECT_OUT;
            token_count++;
            i++;
        }
    }
}
```

```
    } else {
        // Read Word/Argument
        int start = i;
        while (i < len && !isspace(input[i]) && input[i] != '|' && input[i] != '<' && input[i] != '>') {
            i++;
        }
        int word_len = i - start;
        char *word = malloc(word_len + 1);
        strncpy(word, input + start, word_len);
        word[word_len] = '\0';

        tokens[token_count].value = word;
        tokens[token_count].type = TOKEN_WORD;
        token_count++;
    }
}
return token_count;
}
```

```
// 2. PARSER: Validates syntax and generates execution structures
int parse(Token tokens[], int token_count, Command *cmd) {
    // Handle Empty Commands
    if (token_count == 0) {
        printf("[Parser] Empty command detected.\n");
        return 0; // Valid, but nothing to execute
    }

    // Detect Error: Command starts with pipe
    if (tokens[0].type == TOKEN_PIPE) {
        printf("[Syntax Error] Unexpected token '|' at start of command.\n");
        return -1;
    }
}
```

```

for (int i = 0; i < token_count; i++) {
    if (tokens[i].type == TOKEN_WORD) {
        cmd->args[cmd->arg_count++] = tokens[i].value;
    } else if (tokens[i].type == TOKEN_REDIRECT_OUT) {
        // Validate Syntax: ensure filename follows redirection
        if (i + 1 >= token_count || tokens[i + 1].type != TOKEN_WORD) {
            printf("[Syntax Error] Missing output redirection file after '>'.\n");
            return -1;
        }
        cmd->output_file = tokens[++i].value;
    } else if (tokens[i].type == TOKEN_REDIRECT_IN) {
        if (i + 1 >= token_count || tokens[i + 1].type != TOKEN_WORD) {
            printf("[Syntax Error] Missing input redirection file after '<'.\n");
            return -1;
        }
        cmd->input_file = tokens[++i].value;
    } else if (tokens[i].type == TOKEN_PIPE) {
        if (i + 1 >= token_count) {
            printf("[Syntax Error] Missing command after pipe '|'.\n");
            return -1;
        }
        cmd->has_pipe = 1;
    }
}
cmd->args[cmd->arg_count] = NULL; // Null terminate argument array
return 1;
}

```

```

// Memory Cleanup
void free_tokens(Token tokens[], int token_count) {
    for (int i = 0; i < token_count; i++) {
        free(tokens[i].value);
    }
}

int main() {
    char input_buf[256];

    while (1) {

```

```

printf("\nParser Shell > ");
if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

// Remove trailing newline
input_buf[strcspn(input_buf, "\n")] = 0;

if (strcmp(input_buf, "exit") == 0) break;

Token tokens[MAX_TOKENS];
Command cmd = { .arg_count = 0, .input_file = NULL, .output_file = NULL, .has_pipe = 0 };

// Step A: Tokenize
int token_count = tokenize(input_buf, tokens);

// Debug Output: Tokens
printf("--- Debug: Tokenizer Output ---\n");
for (int i = 0; i < token_count; i++) {
    printf("Token [%d]: Value=\"%s\", Type=%d\n", i, tokens[i].value, tokens[i].type);
}

// Step B: Parse & Validate
printf("--- Debug: Parser & Execution Structure ---\n");
int status = parse(tokens, token_count, &cmd);

if (status == 1) {
    printf("Command Executable: \"%s\"\n", cmd.args[0]);
    printf("Arguments Count: %d\n", cmd.arg_count);
    if (cmd.output_file) printf("Redirect Output To: %s\n", cmd.output_file);
    if (cmd.input_file) printf("Redirect Input From: %s\n", cmd.input_file);
    if (cmd.has_pipe) printf("Pipeline Flag: Active\n");
}

free_tokens(tokens, token_count);
}

return 0;

```

```

}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano parser_test.c
abhis@MIKHAIL-37:~$ gcc -g -Wall parser_test.c -o parser_test
abhis@MIKHAIL-37:~$ ^C
abhis@MIKHAIL-37:~$ ./parser_test
```

```
Parser Shell > ls -l > output.txt
--- Debug: Tokenizer Output ---
Token [0]: Value="ls", Type=0
Token [1]: Value="-l", Type=0
Token [2]: Value=">", Type=3
Token [3]: Value="output.txt", Type=0
--- Debug: Parser & Execution Structure ---
Command Executable: "ls"
Arguments Count: 2
Redirect Output To: output.txt
```

```
Parser Shell > ls -l >
--- Debug: Tokenizer Output ---
Token [0]: Value="ls", Type=0
Token [1]: Value="-l", Type=0
Token [2]: Value=">", Type=3
--- Debug: Parser & Execution Structure ---
[Syntax Error] Missing output redirection file after '>'
```

```
Parser Shell > | grep test
--- Debug: Tokenizer Output ---
Token [0]: Value="|", Type=1
Token [1]: Value="grep", Type=0
Token [2]: Value="test", Type=0
--- Debug: Parser & Execution Structure ---
[Syntax Error] Unexpected token '|' at start of command.
```

```
Parser Shell >
--- Debug: Tokenizer Output ---
--- Debug: Parser & Execution Structure ---
[Parser] Empty command detected.
```

```
Parser Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./parser_test
==2556== Memcheck, a memory error detector
==2556== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2556== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2556== Command: ./parser_test
==2556==
```

```
Parser Shell > exit
==2556==
==2556== HEAP SUMMARY:
==2556==    in use at exit: 0 bytes in 0 blocks
==2556==    total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==2556==
==2556== All heap blocks were freed -- no leaks are possible
==2556==
==2556== For lists of detected and suppressed errors, rerun with: -s
==2556== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```